Group Relative PO (GRPO): Replace $V_\phi(\cdot)$ model with a group (per Prompt) baseline which is computed from multiple roll-outs of same prompt.

Set-up: one training step
- → Batch of B prompts $\{x_b\}_{b=1}^{B}$
- → For each prompt, sample K rollouts: $\{y_{b,k}\}_{k=1}^{K} \sim \pi_\theta^{old}(\cdot|x_b)$ (length = T_k)
- → Score each completion using $r_\psi(x,y)$: $r_{b,k} = r_\psi(x_b, y_{b,k})$
- → Compute group-relative advantage:
  - For prompt x_b, build a group-baseline $(\mu_b, A_{b,k})$ as:

$$\mu_b = \frac{1}{K}\sum_{k=1}^{K} r_{b,k} \quad \text{(Group mean)}, \qquad A_{b,k} = r_{b,k} - \mu_b$$

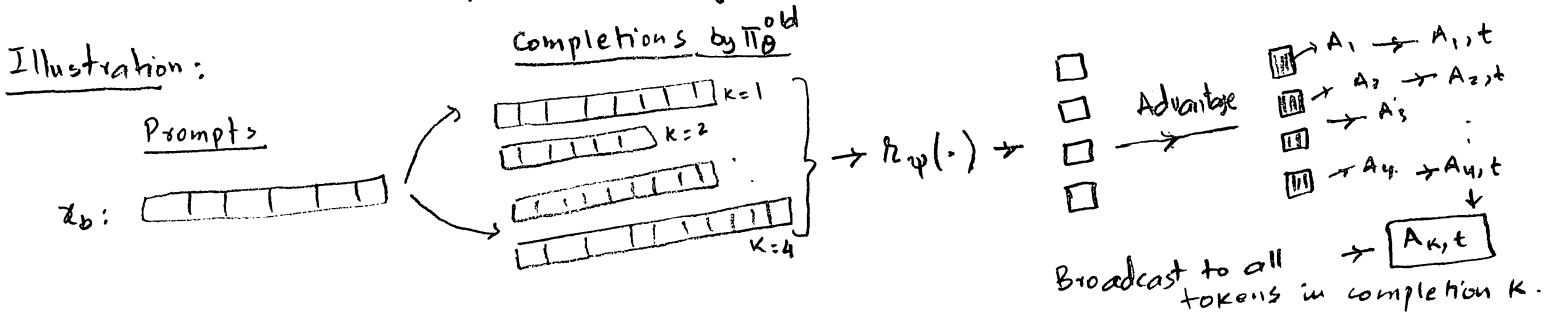
($A_{b,k}$: Advantage for b-th group & k-th roll-out)

Intuition:
- The winner in the group = All those roll-outs $\geq$ Group-mean μ_b
  - ↳ gets positive advantage $A_{b,k}$
- The loser gets -ve $A_{b,k}$.

$\Rightarrow$ This replaces: Token-level $A_{i,t} = V_\phi^{GAE}(s_{i,t+1}) - V_\phi^{old}(s_{i,t})$ of critic/value model

$A_{b,k}$ = called per-prompt relative signal/advantage.

Illustration:



$$L_{GRPO}(\theta) = -\frac{1}{K}\sum_{k=1}^{K}\frac{1}{T_k}\sum_{t=1}^{T_k}\min\left(\rho_{k,t}A_{k,t},\ \text{clip}\left(\rho_{k,t},1-\varepsilon,1+\varepsilon\right)A_{k,t}\right) - \beta\, KL\left(\pi_\theta \,||\, \pi_{ref}\right)$$

($\rho_{k,t}$)

- ↓ for one prompt
- ↓ completion #/index
- ↓ token index
- $A_{k,t}$: sequence-level advantage is broadcast to all tokens in a completion.
- Per-token KLD to SFT-reference

(k = completion # in PPO, we used; i = trajectory index)

$$\rho_{k,t} = \frac{\pi_\theta(\cdot|s_{i,t})}{\pi_\theta^{old}(\cdot|s_{i,t})}$$

Each sequence/completion contributes its mean token loss ($1/T_k$ factor) & all sequences are then averaged.

$\frac{1}{T_k}$ Factor: Causes length-bias/length-invariance problem of GRPO.

Even if two completions have same reward (semantic quality), their contribution in loss is not same.

- Short-sequence ⇒ T_k is small → Per-token weight $\propto \frac{A_k}{T_k}$ is large in magnitude.
- Long-sequence ⇒ T_k is large → " " " " small

" "

Impact: If $A_k > 0$ ⇒ good completion but short-correct answers get stronger per-token push than long-ones.

If $A_k < 0$ ⇒ Bad completion, short bad-answers " "

" " " " "

⇒ Long incorrect response is under-penalized → model starts rambling.

⇒ GRPO tends to produce increasingly long-sequences especially for incorrect outputs: **Optimization Bias in GRPO**

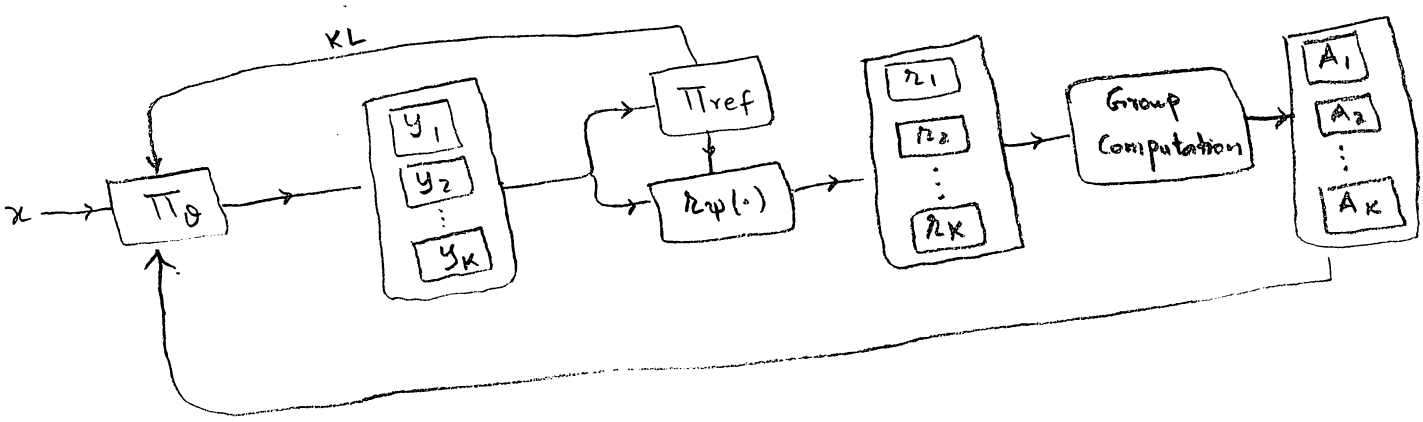
Solution: DAPO: switch to token-level normalization

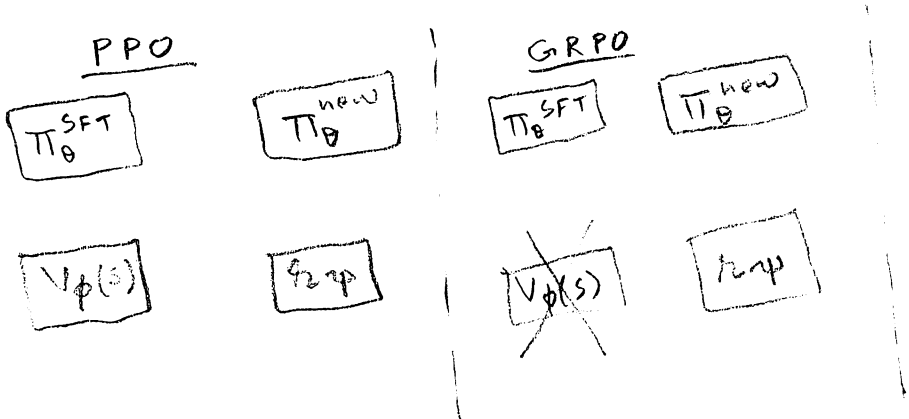
$$L_{DAPO}(\theta) \propto -\frac{1}{\sum_{k=1}^{K} T_k} \sum_k \sum_t \dots$$

→ no per-sequence $\frac{1}{T_k}$ factor.

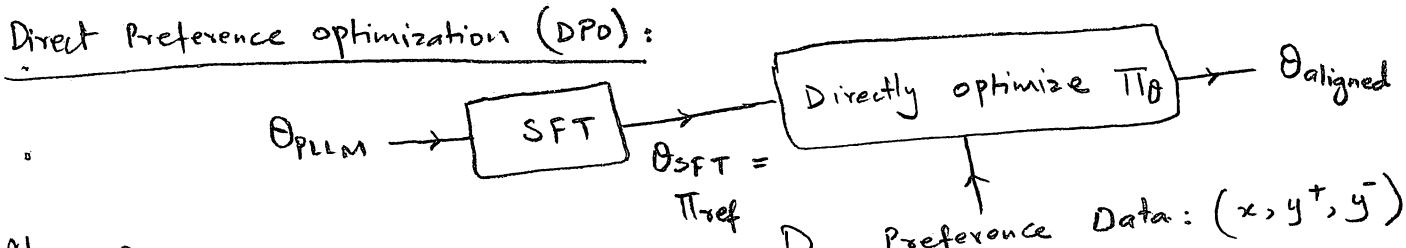
Dr.GRPO: Divide by const. T instead (usually $T = T_{max}$ over all sequences)

$$L_{D\text{-}GRPO}(\theta) \propto -\frac{1}{L} \frac{1}{K} \sum_k \sum_t \dots$$





Direct Preference optimization (DPO):



(y^+ is preferred over y^- by a human annotator)

Idea: Skip $r_\varphi(\cdot)$ + RL loop & use directly D_{Pr} as a supervised signal for training → treat θ_{SFT} as hidden RM

Preference Model - Bradley Terry form: $P(y^+ \succ y^- | x) = \sigma\left(r_\varphi(x, y^+) - r_\varphi(x, y^-)\right)$

$$L_{RM}(\varphi) = -E_{(x,y^+,y^-)}\left[\log\left(\quad\right)\right]$$

DPO directly integrates $y^+ \succ y^-$ into its loss as:

$$L_{DPO}(\theta) = E_{(x,y^+,y^-)}\left[\sigma\left(\beta \underbrace{\log \frac{\pi_\theta(y^+|x)}{\pi_{ref}(y^+|x)}}_{z_1 \text{ (KL term 1)}} - \beta \underbrace{\log \frac{\pi_\theta(y^-|x)}{\pi_{ref}(y^-|x)}}_{z_2 \text{ (KL term 2)}}\right)\right]$$

Increase the log-ratio in favor of y^+ relative to what π_{ref} does, but not infinitely (sigmoid saturates)

$\Rightarrow \sigma(z_1 - z_2)$ same structure as RM sigmoid.

- We can write the log-loss for any single pair as:

$$\ell(\theta) = -\log \sigma(z_1 - z_2) = -\log \sigma\left(\beta \log \frac{\pi_\theta(y^+|x)}{\pi_{ref}(\,\|\,)} - \beta \log \frac{\pi_\theta(y^-|x)}{\pi_{ref}(y^-|x)}\right)$$

$$= -\log\left(\sigma\left[\underbrace{\beta\left(\log \pi_\theta(y^+|x) - \log \pi_\theta(y^-|x)\right)}_{\Delta\theta \,=\, \text{Policy log-ratio}} - \underbrace{\beta\left(\log \pi_{ref}(y^+|x) - \log \pi_{ref}(y^-|x)\right)}_{\Delta_{ref.} \,=\, \text{Reference Correction}}\right]\right)$$

$$= -\log \sigma(z) \quad \text{with} \quad z = \beta(\Delta\theta - \Delta_{ref}) \quad (\Delta_{ref} \text{ is const. wrt } \theta)$$

We can show:

$$\frac{\partial \ell(\theta)}{\partial \log \pi_\theta(y^+|x)} = \frac{\partial \ell}{\partial z} \cdot \frac{\partial z}{\partial \Delta\theta} \cdot \frac{\partial \Delta\theta}{\partial \log(\,\|\,)} = (\sigma(z) - 1) \times \beta \times 1 = \beta(\sigma(z) - 1)$$

&

$$\frac{\partial \ell(\theta)}{\partial \log(y^-|x)} = -\beta(\sigma(z) - 1)$$

During gradient-descent: $\log \pi_\theta \leftarrow \log \pi_\theta - \eta \frac{\partial \mathcal{L}}{\partial \log \pi_\theta}$ → ①

when model does not prefer y^+ over y^-:

⇒ z is small or negative (say $\sigma(z) \approx 0.5$)

then $(\sigma(z)-1) \leq -0.5 < 0$

For good completion y^+: $\frac{\partial \mathcal{L}}{\partial \log \pi_\theta(y^+|x)} = \beta(\sigma(z)-1) < 0$

Then from ①: $\log \pi_\theta(y^+|x)$ increases.

Similarly, for bad completion y^-, $\frac{\partial \mathcal{L}}{\partial \log \pi_\theta(y^-|x)} > 0$

& $\log \pi_\theta(y^-|x)$ decreases

SFT reference Role: $z = \beta(\Delta_\theta - \Delta_{ref})$ if π_θ prefers y^+ over y^- at least as strongly as π_{ref} does, $(\Delta_\theta - \Delta_{ref})$ is large positive ⇒ $\sigma(z) \approx 1$ ⇒ $\sigma(z)-1 \approx 0$ ⇒ gradients ≈ 0 ⇒ Regularization

So, π_θ needs to prefer y^+ more than the π_{ref}; when it starts doing so, no push from π_{ref}.

DPO Issues:

① $\log \pi_\theta(y|x) = \sum_{t=1}^{T_y} \log \pi_\theta(y_t | x, y_{<t})$ → scales roughly with length T_y → Verbosity-Bias → Produces overly-long answers

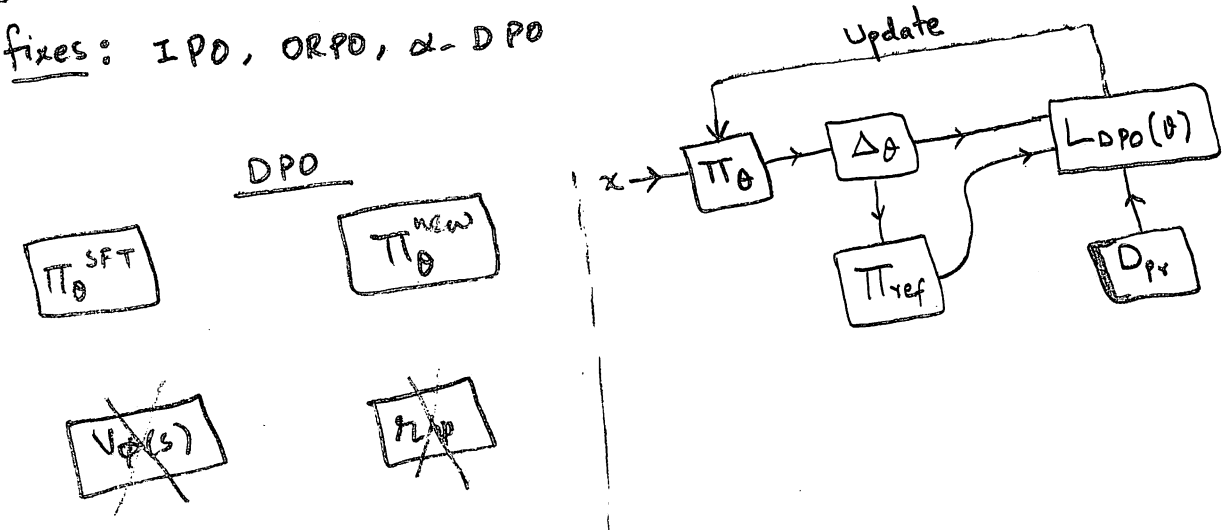
If $T_{y^+} > T_{y^-}$, KL terms in DPO have different lengths ⇒ This difference can either over or under-estimate the preference signal $y^+ > y^-$.

fixes: ① SamPO: Downsample tokens to make contribution length-neutral

② SimPO: use averaged log-prob + reference-free

② Bad π_{ref}: KL regularization pulls everything towards a bad anchor.

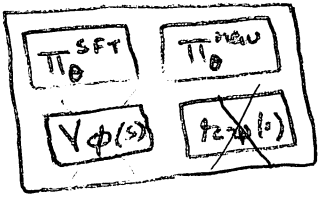
fixes: IPO, ORPO, α-DPO



RL with Verifiable Rewards (RLVR)

Replace $r_{\phi}(\cdot)$ Learned RM with $r_V(\cdot)$ Verifiable RM

Anything else like:



Math: check final answer correctness
Code: run unit tests
Reason: run a checker ...

Keep everything else same:
PPO: use $V_\phi(s)$ as well + GAE
GRPO: use group relative advantage
...

For prompt x (Problem) & completion y (steps + answers):

→ get final answer: final_answer(y)

→ Ground truth = $g(x)$

$$r_{task}(x,y) = \underbrace{1\left[\text{final_answer}(y) = g(x)\right]}_{= r_V(\cdot) \in [0,1]} + \overbrace{\text{optional shaping}}^{\text{Partial credit? + Penalties? etc.}}$$

For PPO: spread it to per-token:

$$r_{i,k,t} = r_{task,i} + \underbrace{r^{KL}_{i,k,t}}_{\longrightarrow -\beta\left(\log \pi_\theta^{old}(y_{i,k,t}|s_{i,k,t}) - \log \pi_{ref}(y_{i,k,t}|s_{i,k,t})\right)}$$

For GRPO: For each prompt x_b & its K-roll-outs:

$$r_{b,k} = r_{task}(x_b, y_{b,k}) - \beta \sum_t \log \pi_\theta^{old}\left(y_{b,k,t} \middle| s_{b,k,t}\right) - \log \pi_{ref}\left(y_{b,k,t} \middle| s_{b,t,t}\right)$$

$$A_{b,k} = r_{b,k} - \mu_b \quad \& \text{ so on.}$$

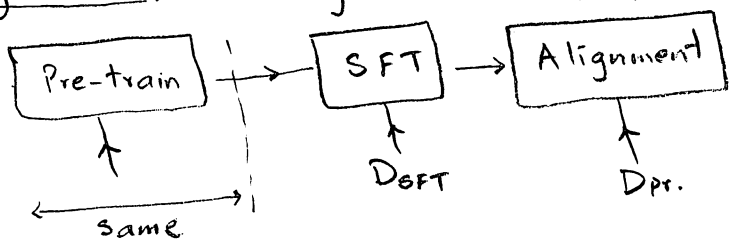
Properties: ① Since $r_V(\cdot) \in [0,1]$ for one (complete) completion, most samples are zero at start $\Rightarrow \nabla = 0$ for long contexts mostly.

② Some prompts are trivial → reward = easy
" " " difficult → " = difficult
} High variance b/w prompts

③ Verifier noise as it might be imperfect

Case studies: — Deepseek R1
— Kimi K1.5
— QWEN 3
} All use RLVR

Reasoning LLMs: Training stack is same



D_{SFT} changes: (x, y)
- y contains detailed multi-step solutions, instead of short helpful answer.
- y has proofs, derivations, codes, comments etc.
- Annotated intermediate steps e.g. "First write the equation as ..., then isolate x, ..."

$\{y_{CoT}\}$ → Long-structured reasoning

$$y_{CoT}: \{\underbrace{\langle \text{Reasoning Tokens} \rangle}_{\text{This portion is called reasoning trace: } \{y_1, \ldots, y_{T_{think}}\}}, \langle \text{Answer Tokens} \rangle\} \leftarrow \text{conceptual split}$$

So, full roll-out is:

$$\left(x, \underbrace{y_{1:T_{think}}}_{\text{Reasoning Trace}}, \langle \text{Answer Tokens} \rangle\right) = \text{one trajectory}$$

In classical CoT, trace is shown to user ("let's think step by step...") while in reasoning LLMs generate an internal CoT ("reasoning tokens") then hide it & show final answer.

- **Reasoning Trace:** $y_{1:T_{think}}$ for a given x
- **" Tokens:** each $y_t \in y_{1:T_{think}}$
- **" Vocabulary:** The subset of token types that are specially x-ic of reas. traces.

$r_\psi(\cdot)$ changes:
- (x, y^+, y^-) changes to reflect correctness of final answer, quality of reasoning & style (non-rambling CoT).
- $r_\psi(\cdot)$ can be learned one with:
  - → **ORM:** 1 reward for final answer
  - → **PRM:** Per-step reward: $r_t = \begin{cases} +1 & \text{step} = \checkmark \\ -1 & \text{step} = \times \\ 0 & \text{otherwise} \end{cases}$
- often **RLVR** is used

- **Self-training / self-verification loops:** model generates many candidate solutions → A verifier filters them → Surviving solutions as new SFT data.

Alignment/RLHF Issues:

① **objective level**:
- we use (SFT + $r_\psi(\cdot)$ + KL) → This is proxy to HHH. During PO, we push towards scalar reward, its mistakes may dominate.
- Preference data is noisy + biased + sparse.
- Reward = helpfulness + λ × harmlessness − β × toxicity + ...
  only one scalar: opaque trade-offs b/w HHH.

② **optimization level**:
- Sequence space is huge but reward is sparse ⇒ ∇'s become noisy & high-variance
- Long reasoning traces: → reward: scalable + automatic + NO humans
- **Overparameterization**: Models learn to exploit quirks in $r_\psi(\cdot)$ (or verifier) due to its enormous size. This is also called reward-hacking.
- **Mode collapse**: Probability mass concentrates on narrow patterns that perform well.
- **KL Spikes**: If we push too hard, we move away from θ_{SFT}.

③ **Length/Verbosity Bias**: RLHF models become over-verbose.

④ **Behavior after training**:
- Jailbreaks
- **OOD Generalization**: Alignment mostly sees nice prompts. In the wild, we may have multi-turn setups + long-horizon tool use + new topics.

Open Directions:

- **Better objectives than PPO** → Direct PO that avoids some pathologies of RLHF
- **Better Reward & Verification** → Combine human preference + rules + automated checks
- **More Robust Optimization** → -DRO type methods
- **Understanding Policy π_θ** → Mechanistic Interpretability & probes on reasoning traces
- **Monitoring Policy π_θ** → Detect reward hacking early

Uses of LLM:

① Inference : Deploy as general-purpose assistant

② SFT Data Creation : Use to create SFT data.

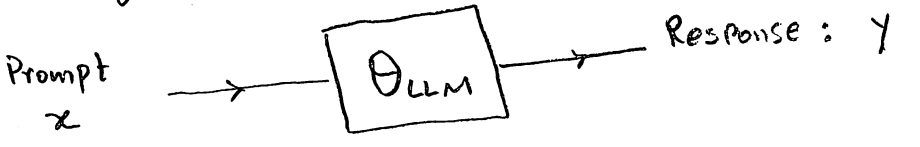
③ RL Roll-outs : K roll-outs of a prompt.

④ Evaluation : Run to check how good it is on a test prompt set. e.g. A/B Testing

⑤ Distillation : Train small S model on reasoning traces of T model.

⑥ Tools & System Building : RAGs + Agents + Pipelines.

In all of these :



Prompt Techniques : How you call the model θ_{LLM} ?

① Reason-enhancing Techniques :

- CoT : break reasoning into steps for logical tasks
- Self-Consistency : Sample multiple CoT y & choose most common
- Tree of thought : Explore multiple reasoning branches
- ReAct : Reasoning + Acting : Combine thought steps with tools | functions | API calls.

② Core Techniques :

- Zero shot prompting : Ask θ_{LLM} to produce y with no examples
- One " " : " " " " " " one "
- Few " " : " " " " " " few "

③ Instruction & Role-based Techniques :

- Instruction Prompting : clearly define the task ("summarize this in one sentence")
- System/Role " : set θ_{LLM} persona ("you are a legal assistant")
- Instruction + Few-shot : Combine both

④ Prompt Composition Techniques :

- Prompt chaining : Use y of one x, as x for another : $x_A \to y_A = x_B \to y_B$
- Dynamic Prompting : Insert context programmatically into x
- Meta " : Ask θ_{LLM} to review its own y

⑤ Multimodal Prompting :

- Image + Text prompting : Text + image e.g. Task = captioning
- Audio/Video " : Voice/video in x e.g. Task = Meeting Notes